TEMA 03 · PROJETO INTEGRADOR

PEDIDOJÁ

Sistema de acompanhamento de pedidos de e-commerce e notificações por Telegram

API REST • PostgreSQL • Telegram Bot API • ViaCEP • Docker • AWS

Integrantes: _____

Professor(a): _____

Instituição / turma: _____

2026

1. Resumo do projeto

O PedidoJá é um sistema acadêmico para registrar clientes, produtos e pedidos de e-commerce, acompanhar as etapas de compra e informar os clientes automaticamente pelo Telegram. O projeto é baseado no Tema 03 do arquivo entregue pelo docente, incluindo os requisitos comuns de API própria, PostgreSQL, migrations, integração com API pública, mensageria via webhook, Docker, CI e plano de publicação em nuvem.

O software contempla API REST em Node.js/Express, interface administrativa responsiva, PostgreSQL, ViaCEP para validar endereço de entrega, Telegram Bot API para envio/recebimento de notificações e um worker para processamento de filas. A arquitetura de nuvem escolhida é AWS EC2, RDS PostgreSQL, Route 53 e HTTPS com Caddy/Let's Encrypt.

2. Problema, justificativa e objetivos

Problema: consumidores precisam acessar o site repetidamente para acompanhar o pedido, e a equipe da loja costuma responder manualmente às mesmas perguntas. Uma solução de mensagens automáticas reduz consultas repetitivas e melhora a transparência do fluxo de atendimento.

Objetivo geral: desenvolver um sistema que centralize pedidos e notifique o cliente quando seu pedido muda de status.

- Cadastrar clientes, produtos, pedidos e seus itens.
- Validar CEP via ViaCEP e preservar o endereço de entrega.
- Aplicar o fluxo recebido → pago → separado → enviado → entregue.
- Vincular usuários ao bot mediante consentimento (opt-in) e registrar mensagens.
- Publicar a solução com banco gerenciado, DNS e HTTPS após configuração das credenciais.

3. Escopo e atores

O administrador manipula os cadastros e os status dos pedidos. O cliente aceita receber avisos no Telegram, recebe mensagens e pode confirmar ciência por botão, além de consultar seu pedido pelo comando /pedido NUMERO. Os atores externos são Telegram e ViaCEP.

Fora do escopo deste MVP: gateway de pagamento, controle de estoque, integração com transportadora e autenticação multiusuário. O status "pago" é uma ação do operador, não uma verificação bancária.

4. Requisitos funcionais

ID	Requisito	Implementação
RF01	Cadastro de clientes/endereço	POST /api/clientes
RF02	Validar CEP de cadastro/entrega	ViaCEP
RF03	Cadastrar/listar catálogo	/api/produtos

RF04	Criar pedidos e itens	/api/pedidos
RF05	Avançar as 5 etapas em ordem	PATCH /api/pedidos/:id/status
RF06	Histórico das transições	historico_status
RF07	Vínculo Telegram via link	/start codigo
RF08	Notificar cada mudança	mensagens + worker
RF09	Receber textos e botões	/webhook/telegram
RF10	Auditar ciclo da mensagem	status_msg
RF11	Painel web responsivo	HTML/CSS/JS
RF12	Webhook idempotente e JSON bruto	webhook_updates
RF13	Consultar pedido pelo bot	/pedido NUMERO
RF14	Indicadores e logs	/api/resumo /api/mensagens

5. Requisitos não funcionais

ID	Característica	Estratégia
RNF01	Segurança	Segredos fora do Git, header de webhook e HTTPS
RNF02	Persistência	PostgreSQL com migrations SQL versionadas
RNF03	Confiabilidade	Fila e tentativas exponenciais para erros transitórios
RNF04	Idempotência	update_id único; ignorar eventos repetidos
RNF05	Nuvem	EC2 + RDS + Route 53 + Caddy
RNF06	Qualidade	Testes unitários e pipeline GitHub Actions
RNF07	Observabilidade	/metrics e perfil Prometheus/Grafana

6. Principais regras de negócio

- O cliente deve iniciar a conversa pelo convite para habilitar o recebimento (opt-in).
- O pedido começa em recebido e avança apenas uma etapa por vez.
- O preço do item é obtido no banco e congelado no momento do pedido.
- O cadastro do pedido revalida o CEP de entrega.
- Convites são gerados aleatoriamente, expiram em 24 horas e são armazenados como hash.
- O usuário só consulta pelo bot os pedidos associados ao próprio chat_id.
- Telegram não permite medir leitura; respondida significa que o cliente clicou no botão.
- Mesmo sem opt-in, a falha de notificação é registrada com motivo no banco.

7. Arquitetura e componentes

Componente	Responsabilidade
Interface web	Cadastros, pedidos, status, logs e indicadores
API REST Express	Valida entradas, executa regras, consulta SQL
Serviço ViaCEP	Consulta endereço via CEP e trata erros

PostgreSQL	Guarda usuários, produtos, pedidos, status e mensagens
Webhook Telegram	Valida cabeçalho, persiste JSON e entrega ao worker
Worker de mensagens	Processa entradas, envios, retentativas e callbacks
Docker Compose	Orquestra aplicação, worker e banco local
AWS + Caddy	Publica API com banco gerenciado e HTTPS

Fluxo lógico: interface → API REST → PostgreSQL. A criação/alteração do pedido registra uma mensagem enfileirada. O worker busca a mensagem, chama a Telegram Bot API e registra o resultado. Para entradas, Telegram → webhook HTTPS → webhook_updates → worker → vinculação ou confirmação → banco de dados.

8. Modelo relacional

Tabelas do domínio: usuarios, clientes, produtos, pedidos, itens_pedido, historico_status. Tabelas de mensageria e segurança: canais_usuario, mensagens, vinculos_pendentes, webhook_updates. Tabela de controle de versionamento: migrations. As relações usam chaves estrangeiras e o status do pedido é um ENUM PostgreSQL.

No modelo, usuarios 1:1 clientes; clientes 1:N pedidos; pedidos 1:N itens_pedido; produtos 1:N itens_pedido; pedidos 1:N historico_status; usuarios 1:N canais_usuario; canais_usuario 1:N mensagens. Os campos de mensagens seguem o Anexo A.5: canal_usuario_id, plataforma, direcao, id_externo, conteudo, status, erro_codigo, erro_descricao, payload_bruto e datas.

9. Integrações externas

9.1 ViaCEP

Consulta HTTPS GET /ws/{cep}/json/. A API normaliza o CEP para oito dígitos, rejeita formatos inválidos, interpreta resposta com erro=true e oferece resposta 503 se o serviço estiver indisponível. Os dados de endereço são armazenados para o cliente e copiados como snapshot para o pedido.

9.2 Telegram Bot API

O bot é criado pelo @BotFather. O usuário abre um deep link com /start <codigo> e o webhook associa chat_id ao usuário válido. Cada status do pedido gera uma mensagem na fila. O worker chama sendMessage, salva message_id em id_externo e exibe botão inline Confirmar ciência. Ao clicar, um callback_query retorna pelo webhook e a mensagem passa a respondida. Recebimento e saída são gravados no banco, assim como os erros 429 e 403.

10. Segurança e resiliência

- As rotas administrativas usam X-Admin-Key definida como segredo fora do repositório.
- O webhook exige X-Telegram-Bot-Api-Secret-Token, com comparação em tempo constante.

- O update_id do Telegram é uma chave única que impede o processamento em duplicidade.
- A fila é persistente e implementa tentativas e espera quando houver limite 429; erros 403 inativam vínculo.
- Tokens são lidos de variáveis de ambiente, não são enviados ao navegador.
- Em produção o RDS fica privado: porta 5432 liberada somente para Security Group da EC2; TLS usando certificado CA.

11. Testes e critérios de aceitação

Teste	Critério de aceitação
Transição de etapas	Não permite saltar status, retorna HTTP 409
Validação CEP	Consulta válido e rejeita inexistente/indisponível
Preço	Recusa preço zero/decimal impróprio; armazena centavos
Webhook falso	Cabeçalho incorreto retorna HTTP 401
Duplicidade	Repetir update_id não registra o mesmo evento duas vezes
Confirmação	Clique do cliente registra respondida
Falhas Telegram	429 programa retry; 403 inativa canal
Consulta protegida	/pedido de outra conta não revela pedido

Verificação executada na preparação: 10 testes unitários passaram usando Node.js 22 e mocks de ViaCEP/Telegram; todos os arquivos JavaScript passaram em verificação de sintaxe. Testes com Docker, PostgreSQL e Telegram reais ainda dependem de execução no ambiente do grupo. Não há evidência de AWS publicada neste documento.

12. Publicação e operações de nuvem

A publicação planejada usa EC2 Linux para os containers api/worker/Caddy e RDS PostgreSQL gerenciado como banco externo. Route 53 resolve o domínio por registro A; Caddy solicita e renova o certificado HTTPS. As regras de entrada da EC2 permitem 80/443 e acesso administrativo restrito; a porta 5432 do banco recebe tráfego somente da EC2.

O projeto prevê controle de custo com AWS Budgets nos patamares US$ 25, US$ 50 e US$ 100. O documento de implantação inclui checklist de prints reais de EC2, RDS, SG, migrations, DNS, certificado, Telegram e orçamento. Os recursos da AWS e a URL final não foram criados automaticamente porque dependem das credenciais e escolhas do grupo.

13. Plano de execução, evidências e limitações

- Executar docker compose local e registrar o log de migrations.
- Criar bot do Telegram e configurar webhook HTTPS com segredo.
- Demonstrar cadastro, pedido, sequência de cinco status e botão confirmando ciência.
- Publicar o site e banco RDS com DNS e certificado válidos, coletando as capturas exigidas.
- Enviar código ao GitHub com commits e tags por encontro efetivamente realizado e release final v1.0.0.

Limitações: não há pagamento automático nem rastreio de transportadora; a autenticação administrativa é simplificada; serviços externos dependem de rede e credenciais; status enviada não prova leitura. Não é correto apresentar URLs, credenciais, resultados ponta a ponta ou prints AWS fictícios.

14. Referências

Material da disciplina. PROJETO INTEGRADOR — TEMAS, p. 1–2 e Anexos A e B. Arquivo fornecido pelo docente.

TELEGRAM. Telegram Bot API. Disponível em: https://core.telegram.org/bots/api.

VIACEP. Webservice CEP e IBGE gratuito. Disponível em: https://viacep.com.br/.

POSTGRESQL. Documentation. Disponível em: https://www.postgresql.org/docs/.

AMAZON WEB SERVICES. Amazon EC2, RDS e Route 53. Disponível em: https://docs.aws.amazon.com/.

CADDY. Automatic HTTPS. Disponível em: https://caddyserver.com/docs/automatic-https.